

操作系统，2026

第3讲 进程的描述与控制

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

进程的定义和描述

02

进程的状态

03

进程控制

03

Linux和Windows的进程

程序的顺序执行和并发执行

- **程序的顺序执行**：计算机系统中只有一个程序在运行，该程序独占系统中的所有资源。一个程序执行完成之后另一个程序才能开始执行
- 顺序执行是单道批处理系统的执行方式，也用于简单的单片机系统

程序的顺序执行和并发执行

- 程序的顺序执行与程序设计中的顺序控制结构不同
- 程序的顺序执行强调的是程序间的执行序列是顺序的。也就是说，系统中任意时刻只有一个程序被载入内存，占有CPU等资源在运行，一个程序执行完了，才能执行另一个程序

程序的顺序执行和并发执行

■ 顺序执行的特征

- **顺序性**：一个程序必须等到前一个程序已经执行完成才能开始执行
- **封闭性**：独占全部资源，计算机的状态只由该程序的控制逻辑所决定
- **可再现性**：初始条件相同则结果相同。如：可通过空指令控制时间关系

程序的顺序执行和并发执行

- **程序的并发执行**：一定时间内，系统中有两个或两个以上的程序同处于开始运行但尚未结束的状态，并且次序不是事先确定的
- 目前主流的操作系统都支持程序的并发执行，引入并发执行的目的是为了**提高资源利用率**

程序的顺序执行和并发执行

■ 并发执行的特征

- **间断(异步)性**：“走走停停”，一个程序可能走到中途停下来，失去原有的时序关系
- **失去封闭性**：共享资源，受其他程序的控制逻辑的影响。如：一个程序写到存储器中的数据可能被另一个程序修改，失去原有的不变特征
- **失去可再现性**：失去封闭性导致失去可再现性；外界环境在程序的两次执行期间发生变化，失去原有的可重复特征
 - 执行时程序推进的时序不可再现——**允许**
 - 执行的结果不可再现——**不可接受**

程序并发执行结果不可再现的示例

- 订票系统，两个终端，运行T1、T2程序

T1:

```
...  
① read(x);  
② if x>=1 then  
③   x:=x-1;  
④ write(x);  
...
```

T2:

```
...  
① read(x);  
② if x>=1 then  
③   x:=x-1;  
④ write(x);  
...
```

- 假设执行顺序为T1①②③T2①②③④T1④，卖出2张票，写回数据库的票款余额只减了1

程序并发执行引发的问题

- 协调各程序的执行顺序——同步
- 多个并发执行的程序共享系统资源，程序之间可能相互影响，甚至影响输出结果——互斥
- 选择哪些，选择多少个程序进入内存执行——作业调度
- 内存中并发执行的程序哪个先执行，哪个后执行——进程调度
- 内存如何有效分配——内存管理

进程的引入

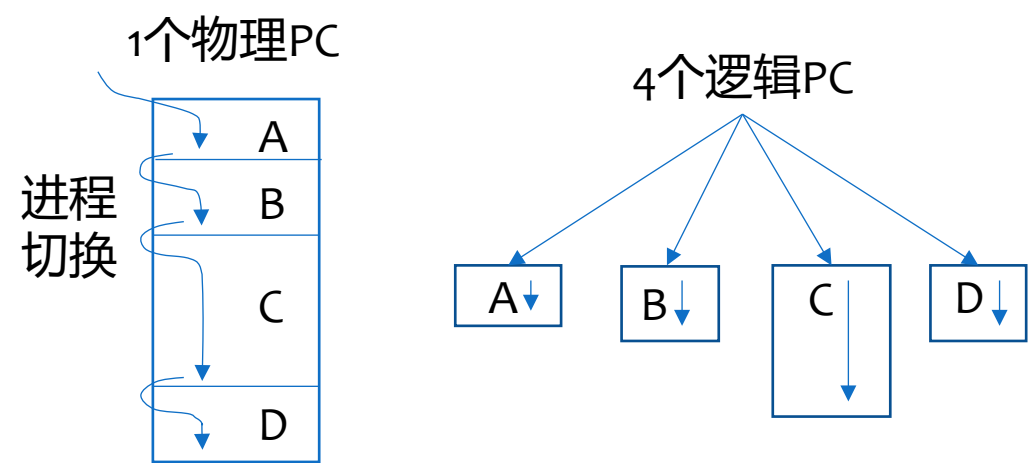
- 进程概念的引入是为了跟踪与描述程序的并发执行
- 当允许程序并发执行时，并发执行的程序可能是同一个程序在不同数据集合上的执行，也可能是不同的程序在不同数据集合上的执行，它们共享系统资源，用程序概念已不能方便地描述程序的并发执行，为了描述程序在并发执行时对系统资源的共享，需要一个描述程序执行时动态特征的概念，所以引入了进程的概念

进程的定义

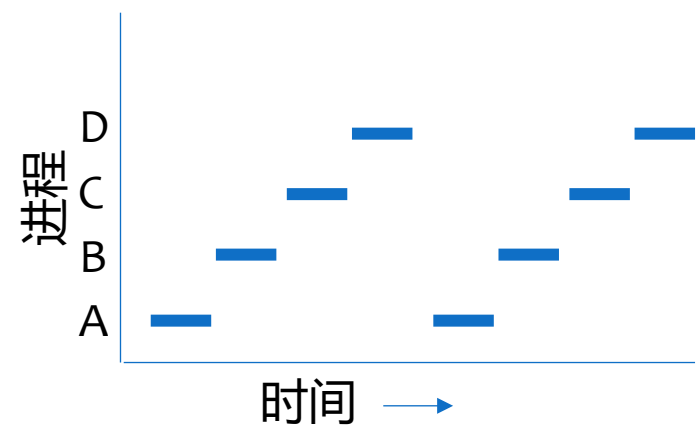
- **进程(Process)**是具有独立功能的程序在某个数据集合上的一次运行活动，是系统进行资源分配和调度的独立单位
- 进程有时也称为**任务(Task)**

进程的定义

■多道程序系统下的进程模型



PC——Program Counter



进程的特征

- **动态性**：进程对应程序的执行
 - 进程是动态产生，动态消亡的
 - 进程在其生命周期内，在三种基本状态之间转换
 - 动态的地址空间
- **独立性**：进程是资源分配的一个独立单位，各进程的地址空间相互独立
- **并发性**：任何进程都可以同其他进程一起向前推进
- **异步性**：每个进程都以其相对独立的、不可预知的速度向前推进
- **交互性**：进程在执行过程中可能与其他进程产生直接或间接的关系
- **结构化**：进程的组成：程序+数据+PCB

进程与程序的区别

- **进程是动态的，程序是静态的**：程序是有序代码的集合；进程是程序的执行。通常进程不可在计算机之间迁移；而程序通常对应着文件、静态和可以复制
- **进程是暂时的，程序的长久的**：进程是一个状态变化的过程，程序可长久保存
- **进程与程序的组成不同**：进程的组成包括程序、数据和进程控制块(即进程状态信息)
- **进程与程序的对应关系**：通过多次执行，一个程序可对应多个进程；通过调用关系，一个进程可包括多个程序

进程控制块

- **进程控制块(PCB, process control block)**是操作系统为了管理进程而设置的一个专门的数据结构，用它来记录进程的外部特征，描述进程的运动变化过程
- 系统利用PCB来控制和管理进程，所以PCB是系统感知进程存在的唯一标志
- 进程与PCB是一一对应的

进程控制块的内容

■ 进程描述信息：

- 进程标识符(process ID), 唯一, 通常是一个整数
- 进程名, 通常基于可执行文件名
- 用户标识符(user ID)
- 进程组 (process group)

■ 进程控制信息：

- 当前状态
- 优先级(priority)
- 代码执行入口地址
- 程序的外存地址
- 运行统计信息(执行时间、页面调度)
- 进程间同步和通信
- 阻塞原因

进程控制块的内容

■ 资源占用信息：

- 虚拟地址空间的现状
- 打开文件列表

■ CPU现场信息：

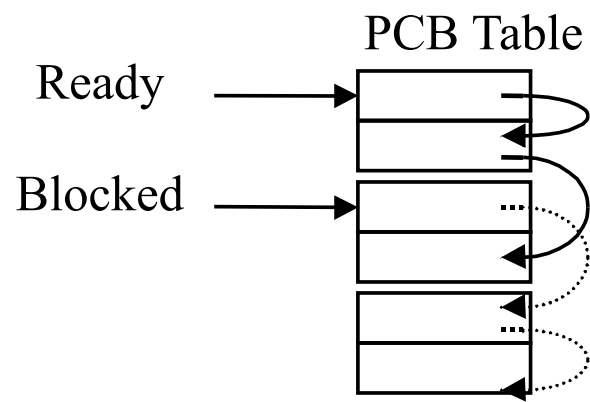
- 寄存器值(通用、程序计数器PC、状态PSW，地址包括栈指针)
- 指向赋予该进程的段/页表的指针

PCB表的组织方式

- 系统把所有PCB组织在一起，并把它们放在内存的固定区域，就构成了PCB表，或称为进程表(process table)
- PCB表的大小决定了系统中最多可同时存在的进程个数，称为系统的并发度
- PCB表的组织方式
 - 链接结构
 - 索引结构

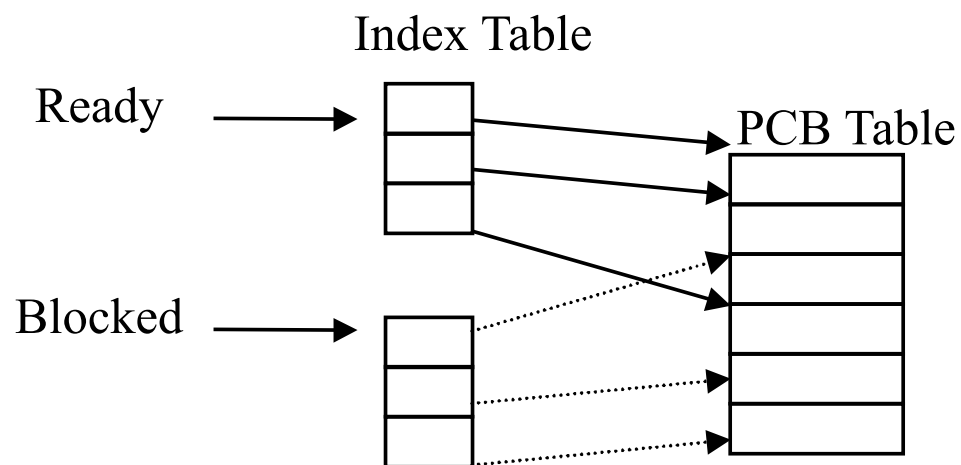
PCB表的组织方式

- **链接结构**：同一状态的进程其PCB成一链表，多个状态对应多个不同的链表
- 各状态的进程形成不同的链表：就绪链表、阻塞链表



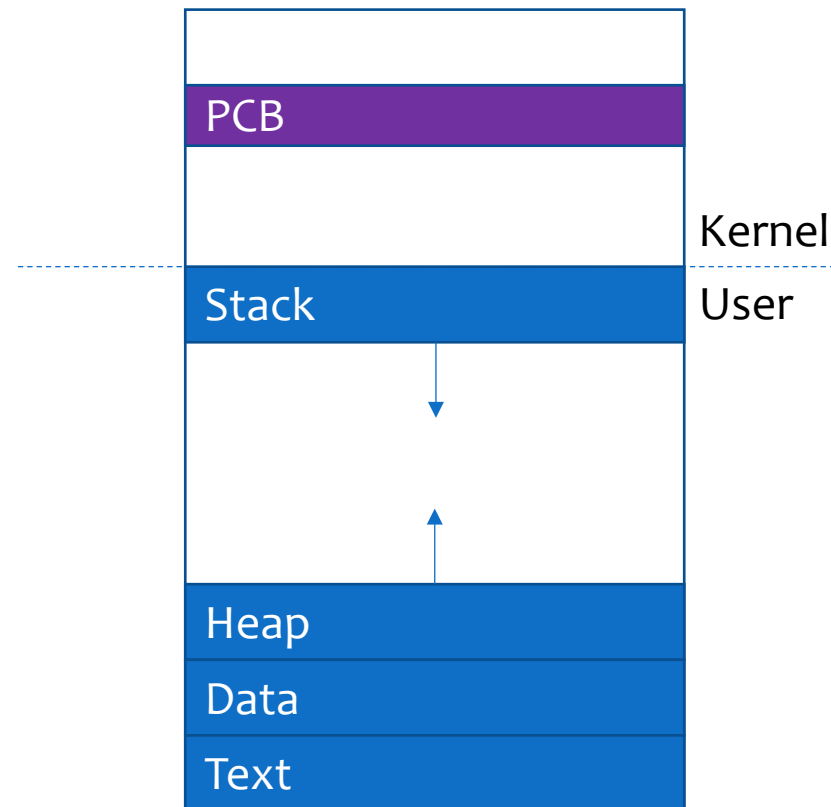
PCB表的组织方式

- **索引结构**：同一状态的进程归入一个index表(由index指向PCB)，多个状态对应多个不同的index表
- 各状态的进程形成不同的索引表：就绪索引表、阻塞索引表



进程映象

- 用户程序(代码)
- 用户数据
- 堆
 - 用于动态内存分配
- 栈
 - 用于过程调用和参数传递
- 进程控制块PCB (进程属性)
 - 用户进程不能直接访问、修改自己的PCB



进程上下文

- 进程上下文(context)是对进程执行过程的描述，由进程的用户地址空间内容、硬件寄存器内容及与该进程相关的核心数据结构组成
- 用户级上下文：进程的用户地址空间(包括用户栈各层次)，包括用户正文段、用户数据段和用户栈；
- 寄存器级上下文：程序寄存器、处理机状态寄存器、栈指针、通用寄存器的值；
- 系统级上下文：
 - PCB和资源表格
 - 核心栈(进程在调用系统核心过程时需要)

操作系统内核和进程的关系

- OS内核不是一个进程，其执行不被调度
- OS和进程的关系：
 - OS不作为进程地址空间的一部分，如传统方法
 - OS作为进程地址空间的一部分，如早期UNIX
 - OS功能分别在核心和系统服务进程中，只有OS核心作为进程地址空间的一部分，如Windows、Linux

下述对进程的描述，错误的是

- ☐ A 进程是动态的
- ☐ B 进程是有生命周期的
- ☐ C 进程执行需要CPU
- ☒ D 进程是指令的集合

提交

大纲

01

进程的定义和描述

02

进程的状态

03

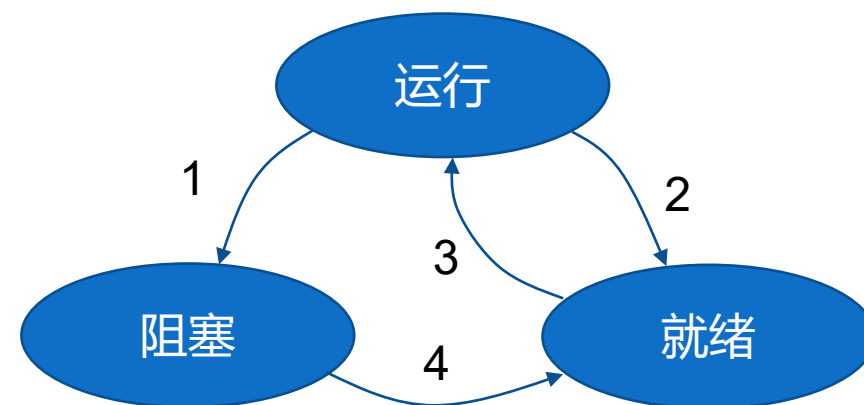
进程控制

03

Linux和Windows的进程

进程的三种基本状态

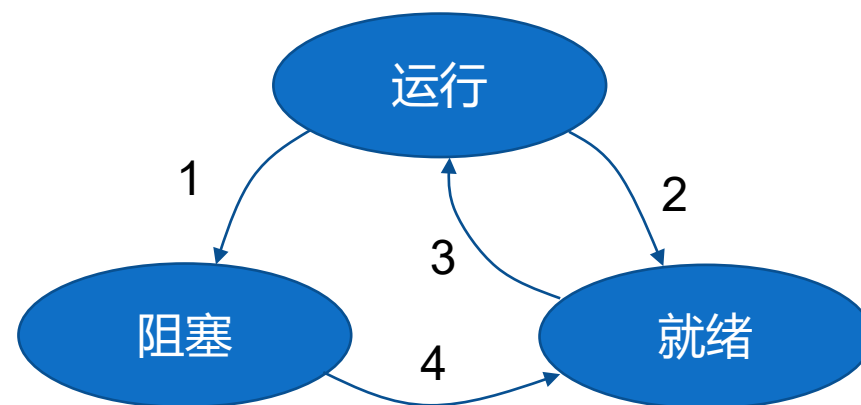
- **运行态(Running)**: 进程占有CPU, 并在CPU上运行
- **就绪态(Ready)**: 一个进程已经具备运行条件, 但由于无CPU暂时不能运行的状态(当调度给其CPU时, 立即可以运行)
- **阻塞态(Blocked)**: 指进程因等待某种事件的发生而暂时不能运行的状态(即使CPU空闲, 该进程也不可运行)



进程的三种基本状态

■ **进程状态转换**：在进程运行过程中，由于进程自身进展情况及外界环境的变化，这三种基本状态可以依据一定的条件相互转换

1. 运行—阻塞
2. 运行—就绪
3. 就绪—运行
4. 阻塞—就绪



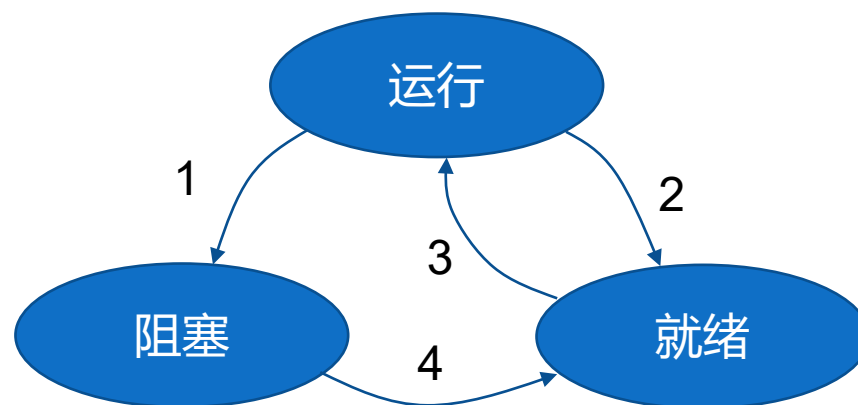
进程的三种基本状态

1. 运行—阻塞

- 当一进程必须等待如下事件时
 - 对一资源的访问尚不能进行
 - 初始化I/O且必须等待结果
 - 等待某一进程提供输入(IPC)

2. 运行—就绪

- 运行进程用完了时间片
- 因为一高优先级进程进入就绪状态，运行进程被中断



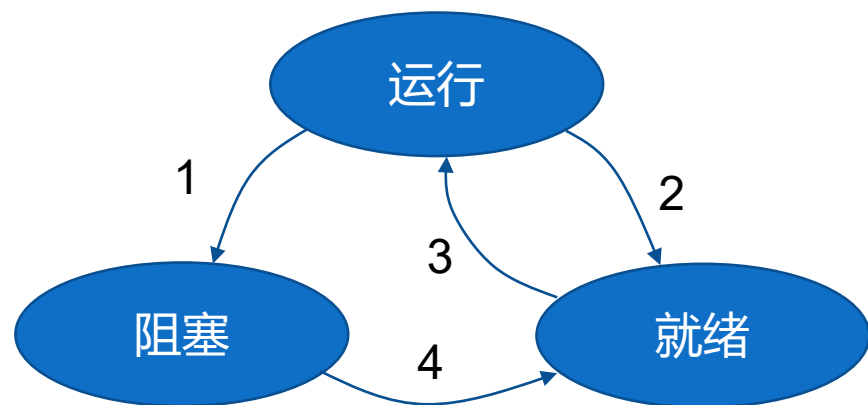
进程的三种基本状态

3. 就绪—运行

- 调度程序选择该进程运行

4. 阻塞—就绪

- 所等待的事件发生



其他状态

- 新建状态(new)
- 终止状态(terminated)
- 挂起(suspended)状态

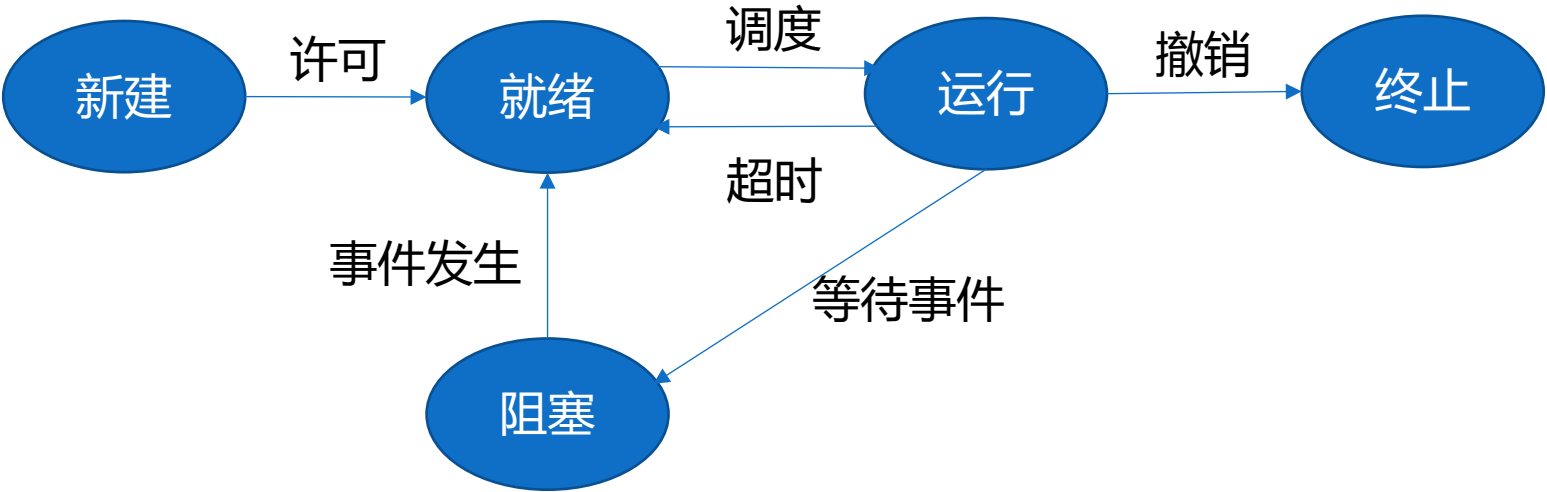
其他状态

- 新建状态(new状态)
- OS已完成成为创建(create)一个进程所必要的工作
 - 已构造了进程标识符
 - 已创建了管理进程所需的表格
- 但还没有许可(admit)执行该进程
 - 因为资源有限

其他状态

- 终止状态(terminated状态)
- 进程撤销(destroy)后进入该状态
- 它不再有执行资格
- 表格和其它信息暂时由辅助程序保留
 - 例如为处理用户帐单而累计资源使用情况的财务程序
- 当数据不再需要后，进程(和它的表格)被删除

五状态进程模型



其他状态

- **挂起状态(suspended状态)**: 进程暂时被淘汰出内存的状态, 也称为**换出状态(swapped out)**
- **可能引入挂起状态的原因**:
 - **用户的请求**: 可能是在程序运行期间发现了可疑的问题, 需要暂停进程
 - **负载调节的需要**: 有一些实时的任务非常重要, 需要得到充足的内存空间, 此时需要把非运行态的任务进行挂起, 优先使得实时任务执行
 - **定时任务**: 一个进程可能会周期性的执行某个任务, 那么在一次执行完毕后挂起而不是阻塞, 这样可以节省内存
- **挂起状态与原有的阻塞和就绪状态结合为阻塞挂起状态和就绪挂起状态**

七状态进程模型

- 就绪状态(Ready): 进程在内存且可立即进入运行状态
- 阻塞状态(Blocked): 进程在内存并等待某事件的出现
- 阻塞挂起状态(Suspended and Blocked): 进程在外存并等待某事件的出现
- 就绪挂起状态(Suspended and Ready): 进程在外存, 但只要进入内存, 即可运行

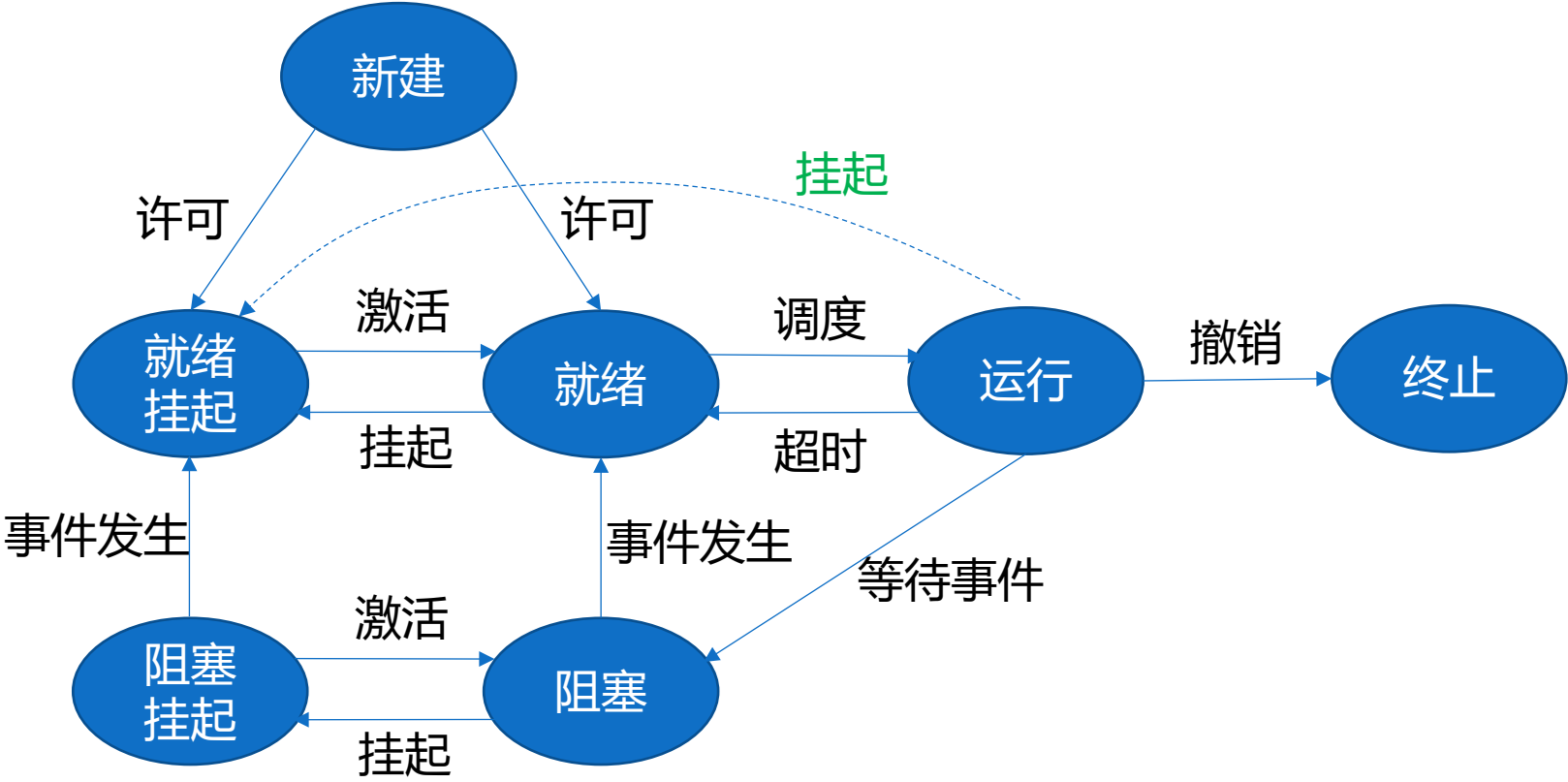
七状态进程模型

- **挂起(Suspend)**: 指把一个进程从内存转到外存, 可能有以下几种情况:
 - **阻塞→阻塞挂起**: 没有进程处于就绪状态或就绪进程要求更多内存资源时, 发生这种转换, 以提交新进程或运行就绪进程
 - **就绪→就绪挂起**: 当有高优先级阻塞(系统认为会很快就绪的)进程和低优先级就绪进程时, 系统会选择挂起低优先级就绪进程
 - **运行→就绪挂起**: 对抢占式系统, 当有高优先级阻塞挂起进程因事件出现而进入就绪挂起时, 系统可能会把运行进程转到就绪挂起状态

七状态进程模型

- **激活(Activate)**: 指把一个进程从外存转到内存; 可能有以下几种情况:
 - **就绪挂起→就绪**: 没有就绪进程或挂起就绪进程优先级高于就绪进程时, 发生转换
 - **阻塞挂起→阻塞**: 当一个进程释放足够内存时, 系统会把一个高优先级阻塞挂起(系统认为会很快出现所等待的事件)进程

七状态进程模型



下述进程状态变化不可能发生的是

☐ A 阻塞→运行

☒ B 就绪→阻塞

提交

在下述系统调用中，不会导致进程阻塞的是_____。

- ☐ A 读/写文件
- ☒ B 获得进程ID
- ☐ C 申请内存
- ☐ D 发送消息

提交

下述描述，正确的是

- ☒ A 进程获得CPU运行是通过调度得到的
- ☐ B 单CPU的系统中，任何时刻都有一个进程处于运行状态
- ☐ C 时间片用完，进程状态变为阻塞
- ☒ D 等待的事件发生，进程状态变为就绪

提交

大纲

01

进程的定义和描述

02

进程的状态

03

进程控制

03

Linux和Windows的进程

进程控制的功能

- 进程控制，就是操作系统使用一些具有特定功能的程序段来创建与撤消进程以及完成进程状态的转换，从而达到多个进程高效率地并发执行和协调，实现资源共享
- 进程控制主要体现在进程的创建与撤销，阻塞与唤醒，挂起与激活□
- 操作系统对进程的控制是依据用户命令和系统状态来决定的。用户可在一定程序上对进程的状态进行控制
- 进程控制由具有特定功能的原语完成

进程控制的功能

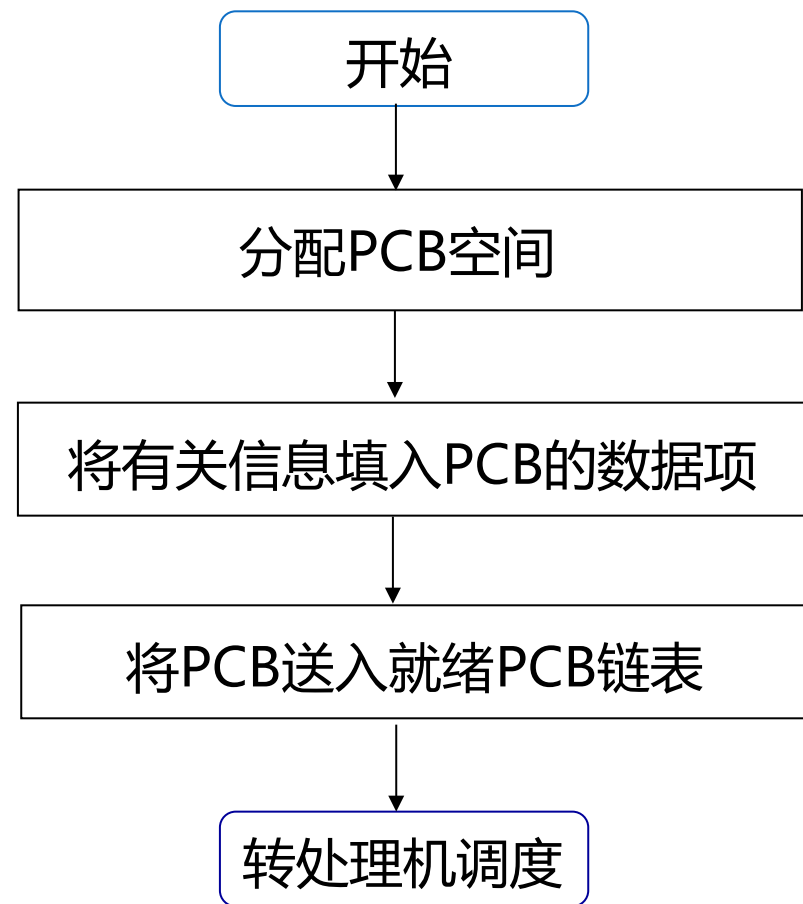
- **原语(primitive)**: 由若干条指令构成的“**原子操作(atomic operation)**”过程，作为一个整体而不可分割——要么全都完成，要么全都不做。许多系统调用是原语
- **注意**: 并非系统调用都是原语。进程A调用read(), 因无数据而阻塞, 在read()里未返回。然后进程B调用read(), 此时read()被重入。系统调用不一定一次执行完并返回该进程, 有可能在特定的点暂停, 而转入到其他进程

创建原语

■ 进程创建的原因

- 系统初始化
- 执行一个创建进程的系统调用
- 用户请求创建一个新进程
- 批处理作业的初始化

- 进程的存在以PCB为标志，所以创建新进程的主要任务就是为进程建立PCB，将调用者提供的有关信息填入PCB的各数据项中

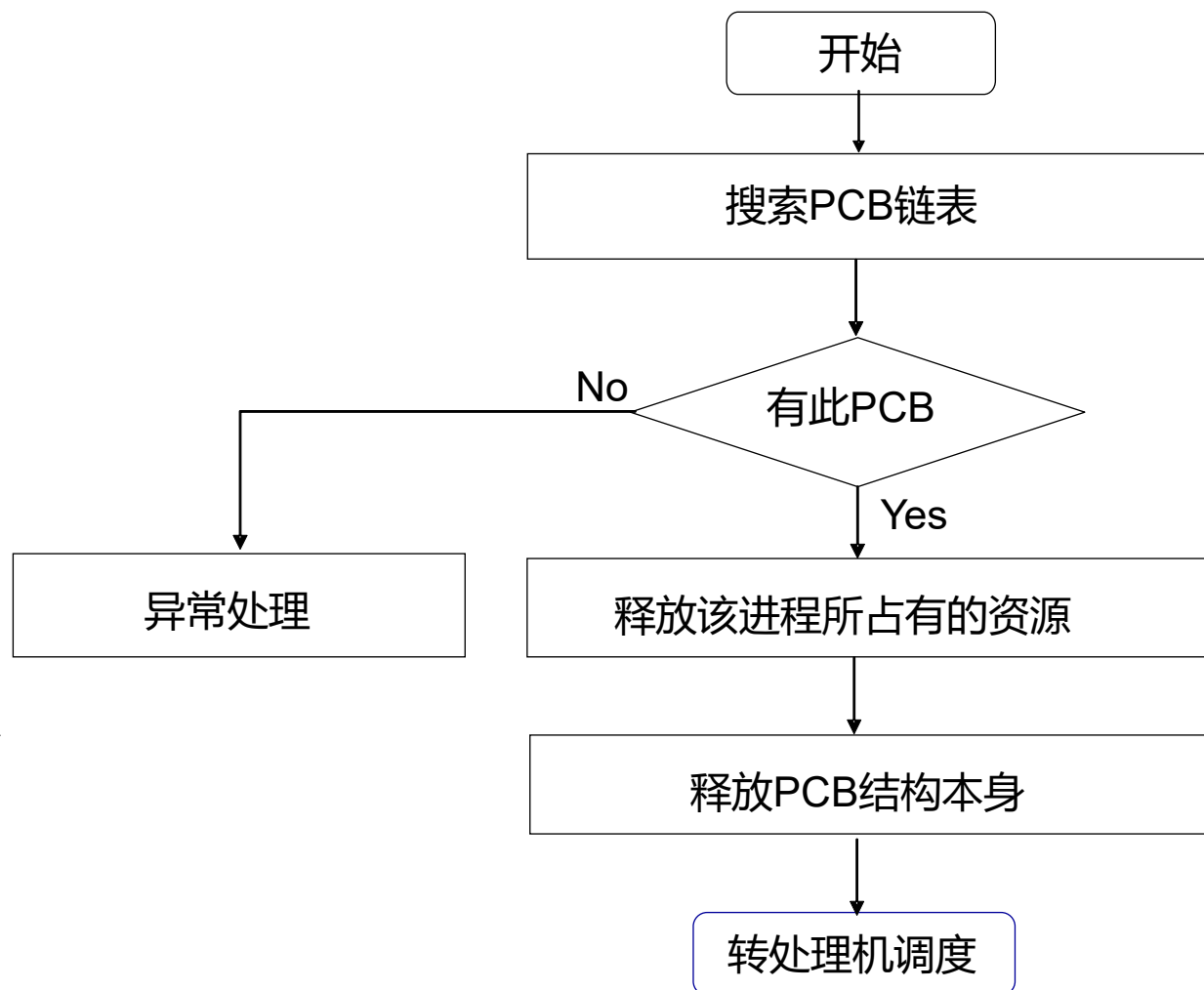


撤销原语

■ 导致进程被撤销的原因

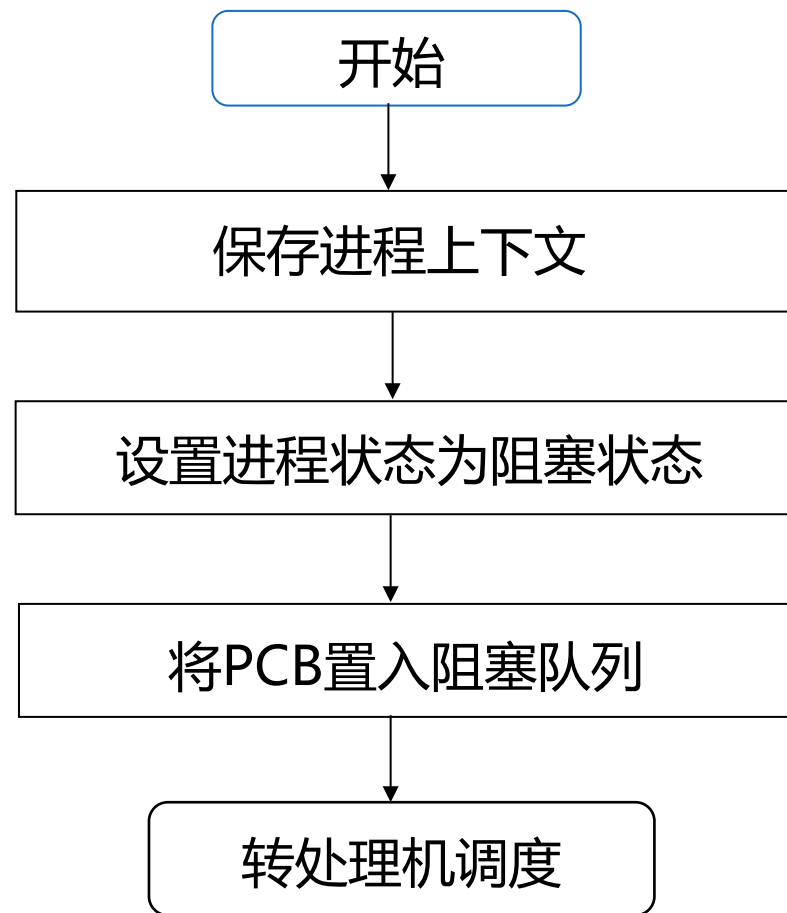
- 正常退出(自愿的)
- 出错退出(自愿的)
- 严重错误(非自愿的)
- 被其他进程杀死(非自愿的)

■ 进程终止之后，系统应及时回收它占有的资源



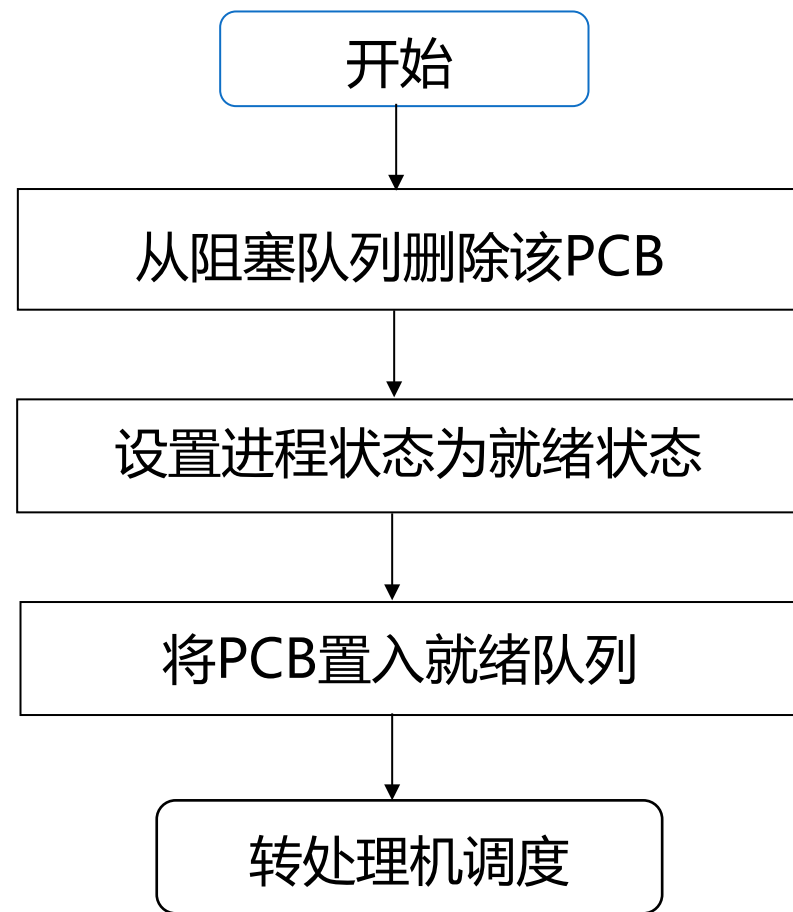
阻塞原语

- 实现进程从运行状态到阻塞状态的转换



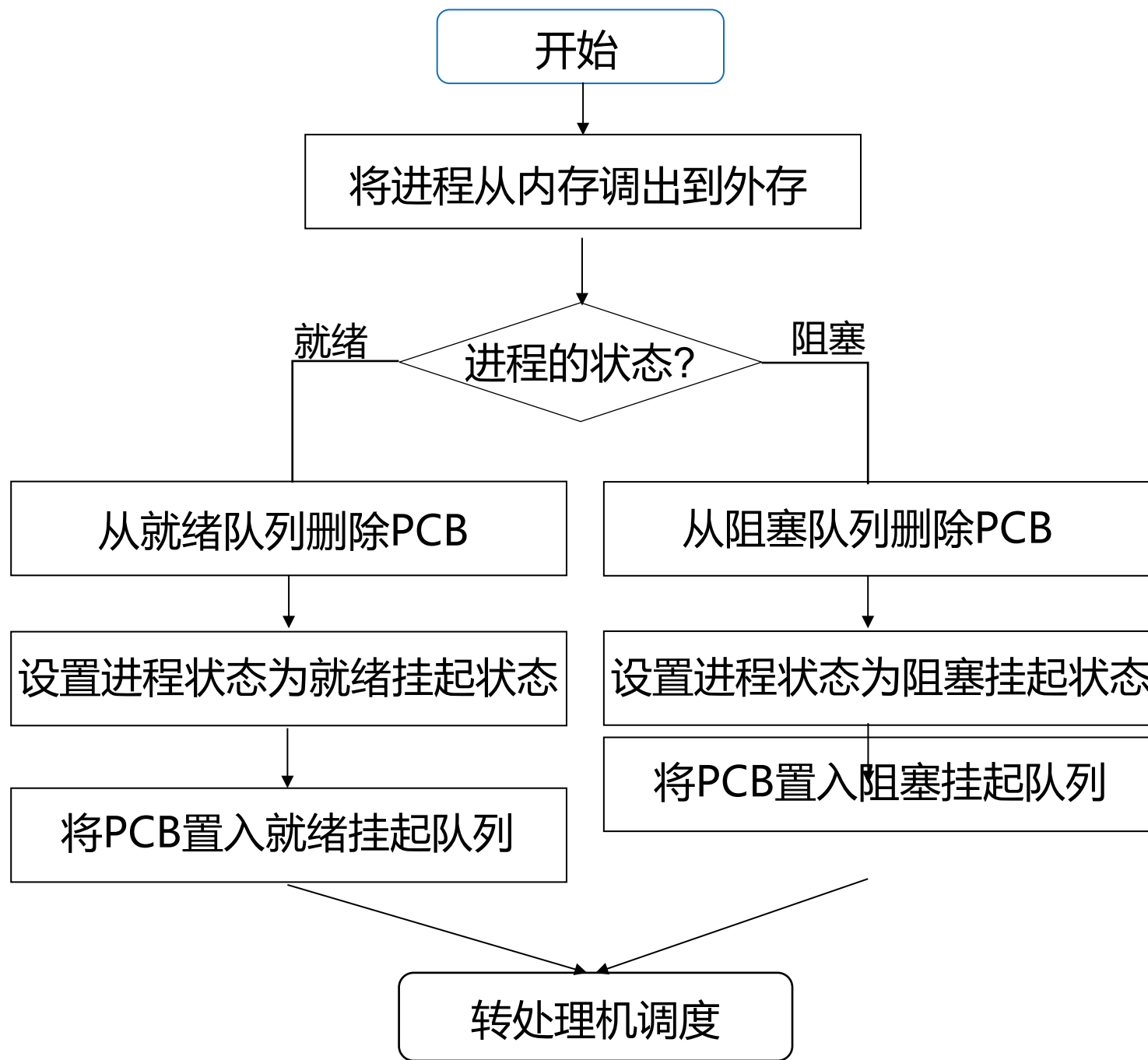
唤醒原语

- 实现进程从阻塞状态到就绪状态的转换



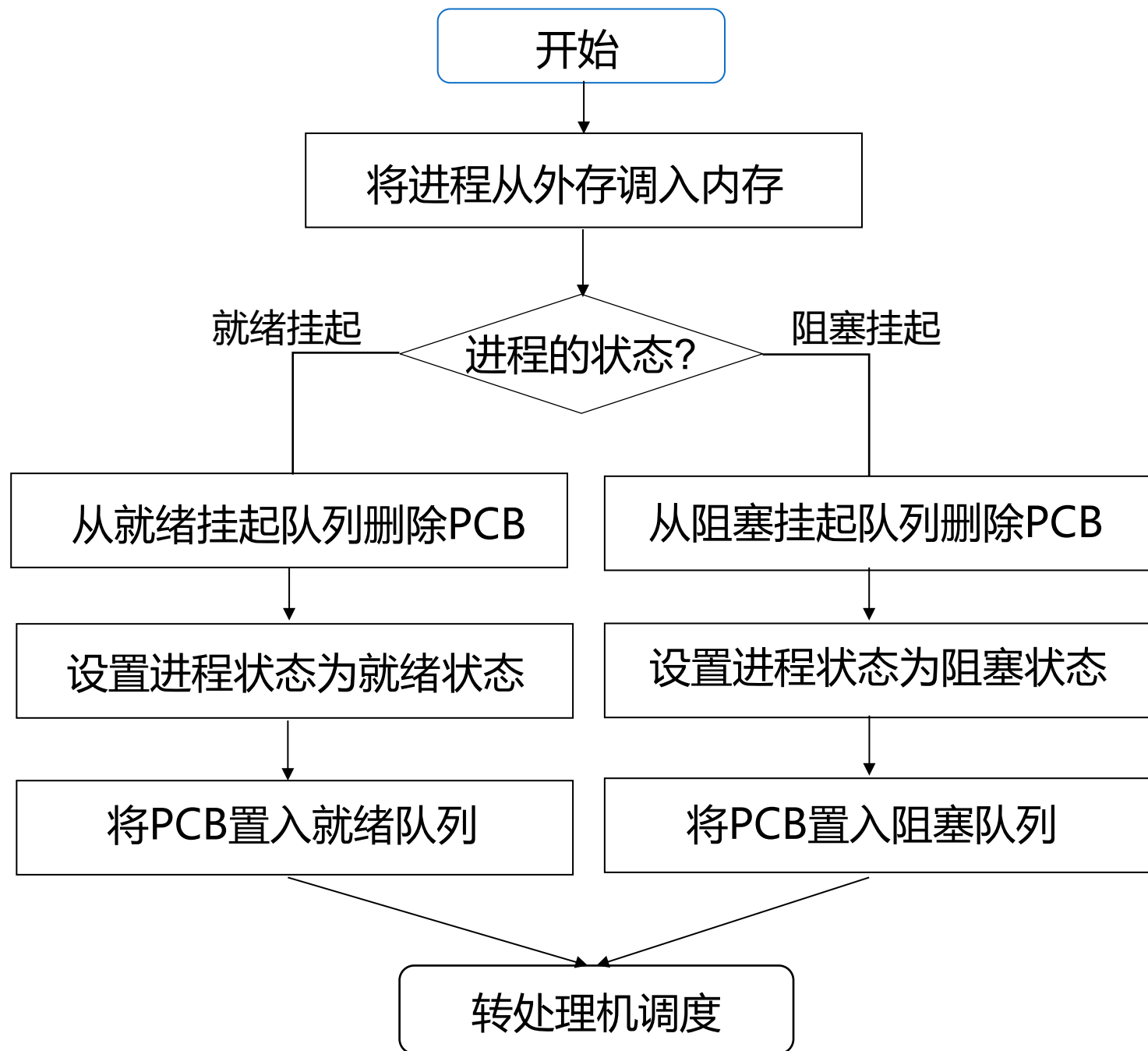
挂起原语

- 实现进程从阻塞或就绪状态到阻塞挂起或就绪挂起状态的转换



激活原语

- 实现进程从阻塞挂起到就绪挂起或就绪挂起到阻塞或就绪状态状态的转换



下述关于原语描述，正确的是

- ☒ A 原语的主要作用是为了实现进程的控制和通信
- ☐ B 为了保证原子性，原语只能包含一条指令
- ☒ C 原语是不可中断执行的操作系统过程
- ☐ D 操作系统的系统调用都是原语

提交

大纲

01

进程的定义和描述

02

进程的状态

03

进程控制

03

Linux和Windows的进程

Linux进程描述符

- Linux的进程控制块是内核数据结构task_struct，Linux将其称为[进程描述符](#)
 - 数据结构：struct task_struct
 - 定义位置：include/linux/sched.h
- 进程描述符向量结构
 - 数据结构：task[NR_TASKS]
 - 定义位置：include/linux/sched.h
 - 定义格式：

```
#define NR_TASKS 512  
struct task_struct *task[NR_TASKS] = {&init_task}
```
 - 全局变量NR_TASKS记录系统可容纳的进程数，默认大小是512

Linux进程描述符

- **进程描述符**结构大约占1000个字节，成员非常多，包括：
 - 进程状态信息(state, flags, ptrace)
 - 调度信息(static_prio, normal_prio, run_list, array, policy)
 - 内存管理(mm, active_mm)
 - 进程状态位信息(binfmt, exit_state, exit_code, exit_signal)
 - 身份信息(pid, tgid, uid, suid, fsuid, gid, egid, sgid, fsgid)
 - 家族信息(real_parent, parent, children, sibling)
 - 进程耗间信息(realtime, utime, stime, starttime)
 - 时钟信息(it_prof_expires, it_virt_expires, it_sched_expires)
 - 文件系统信息(link_count, fs, files)
 - IPC信息(sysvsem, signal, sighand, blocked, sigmask, pending)

Linux进程描述符

- 在Linux内核中，所有进程用一个双向循环链表联系起来形成一个[进程链表\(process list\)](#)，链表的前后指针为PCB中的struct list_head tasks中的prev和next成员
- 链表的表头为init_task进程控制块

Linux进程描述符

- 当调度程序需要把CPU分配给一个新的进程时，只需考虑就绪进程，即状态为TASK_RUNNING的进程，这些进程通过进程控制块中的struct list_head run_list成员组织成双向链表，称之为可运行队列
- 链表的表头也是init_task进程控制块

Linux进程标识

- 成员名：pid_t pid
- 功能
 - 内核通过pid标识每个进程
 - pid与进程描述符之间有严格的一一对应关系
- 数据类型说明
 - pid_t实际上是一个int类型
 - 取值范围：0 ~ 32767
 - 获取进程pid
 - ps命令
 - 访问/proc/pid
 - getpid()

通过ps命令获取进程信息

■ 功能：查看系统中正在运行的进程

■ 语法：

```
ps [-ef][-n name][-t ttys][-p pids][-u users][-groups]
```

进程号

进程的内存占用率

进程使用实际内存的大小

进程的状态

进程已占用的时间

用户ID

进程的CPU占用率

进程使用虚存的大小

启动进程的终端号

进程开始的时间

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.1	0.0	2808	1684	?	Ss	21:30	0:00	/sbin/init
root	2	0.0	0.0	0	0	?	S	21:30	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	21:30	0:00	[migration/0]
root	4	0.0	0.0	0	0	?	S	21:30	0:00	[ksoftirqd/0]
root	5	0.0	0.0	0	0	?	S	21:30	0:00	[watchdog/0]
root	6	0.0	0.0	0	0	?	S	21:30	0:00	[migration/1]
root	7	0.0	0.0	0	0	?	S	21:30	0:00	[ksoftirqd/1]

Linux的进程状态

■ include/linux/sched.h

```
volatile long state; /* -1 unrunnable, 0 runnable, >0 stopped */
```

```
#define TASK_RUNNING          0
#define TASK_INTERRUPTIBLE    1
#define TASK_UNINTERRUPTIBLE  2
#define TASK_ZOMBIE           4
#define TASK_STOPPED          8
#define TASK_EXCLUSIVE        32
```

TASK_SWAPPING为16, 现已废弃

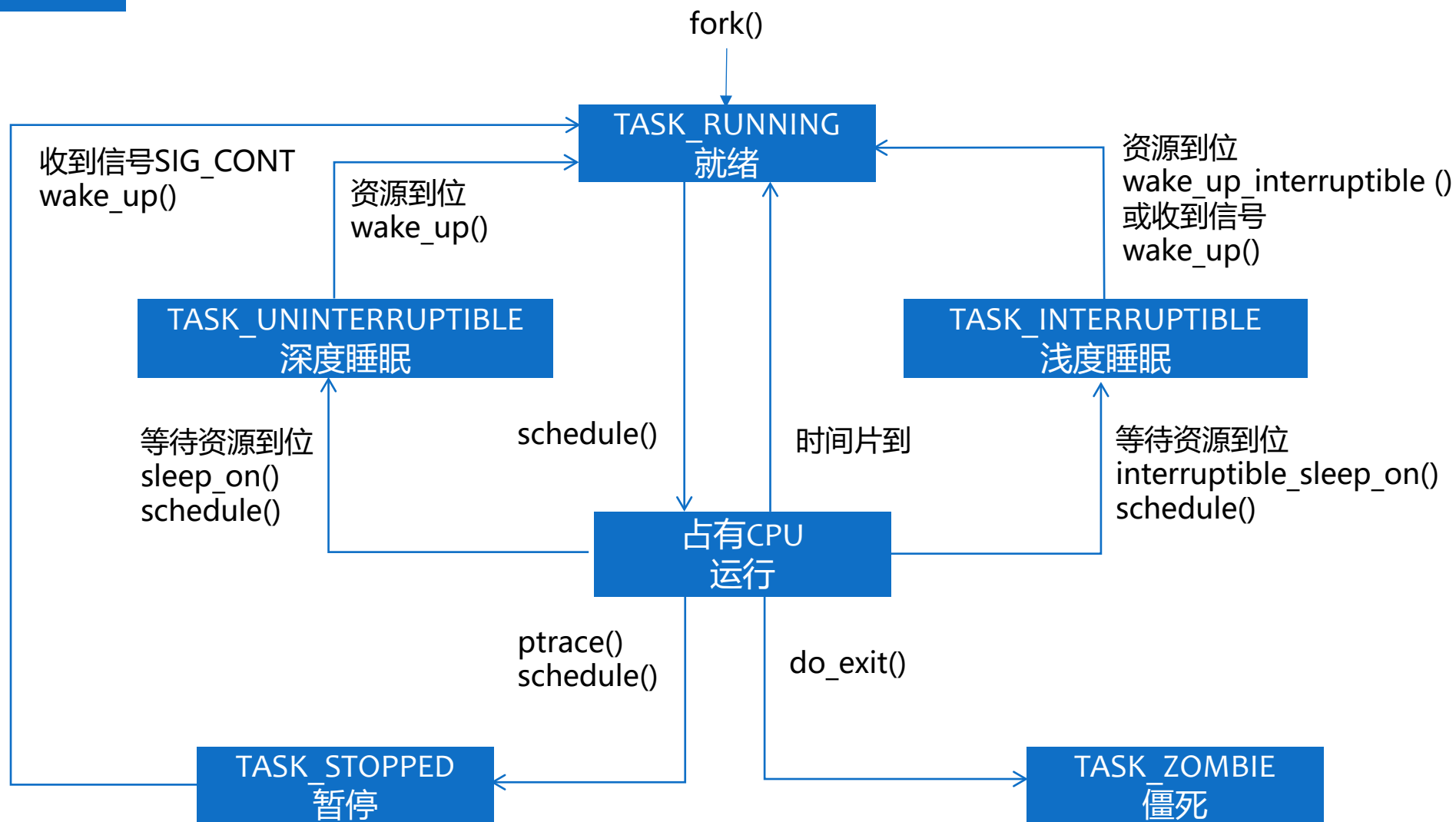
Linux的进程状态

- **可运行态(TASK_RUNNING)**: 表示进程具备运行的资格, 正在运行或等待被调度执行。进程控制块中有一个run_list成员, 所有处于TASK_RUNNING状态的进程都通过该成员链在一起, 称之为可运行队列
- **浅度睡眠态(TASK_INTERRUPTIBLE)和深度睡眠态(TASK_UNINTERRUPTIBLE)**: 两种状态均表示进程处于等待状态。TASK_INTERRUPTIBLE除了资源满足时可以被唤醒外, 还可以被其他进程通过信号唤醒, 而TASK_UNINTERRUPTIBLE不可被其他进程通过信号唤醒

Linux的进程状态

- **暂停状态(TASK_STOPPED)**: 进程处于暂停状态，主要用于调试目的。
如正在运行的进程收到SIGSTOP信号将进入TASK_STOPPED状态
- **僵死状态(TASK_ZOMBIE)**: 表示进程已经结束运行并释放了大部分占用的资源，但task_struct结构还未被释放

Linux的进程状态



POSIX进程控制

■创建进程

- fork——创建新进程，复制现有进程的上下文
- exec——加载新程序并覆盖自身
- 可继承(inherit)：子进程可以从父进程中继承用户标识符、环境变量、打开文件、文件系统的当前目录、控制终端、已经连接的共享存储区、信号处理例程入口表等
- 不可继承：进程标识符，父进程标识符

POSIX进程控制

- `fork` 创建一个新进程（子进程），子进程是父进程的精确复制
- 父子进程的 `fork()` 返回值不同，在子进程中返回为 0；在父进程中，返回子进程的标识
- `fork()` 创建子进程之后，执行返回父进程，或调度切换到子进程以及其他进程
- `getppid()`——返回 Parent PID

POSIX进程控制

- `exec` 用一个新进程覆盖调用进程。它的参数包括新进程对应的文件和命令行参数
- 成功调用时，不再返回；否则，返回出错原因

POSIX进程控制

■ POSIX进程创建

program_a

```
int main()
{
    .....
    if ((child = fork()) == -1) {
        //创建失败
    }
    if (child == 0) {
        //子进程
        if (execlp("program_b", ...) == -1) {
            //加载程序失败
        }
    }
    //父进程
    .....
}
```

program_b

```
int main()
{
    //子进程要执行的程序
}
```

POSIX进程控制

■终止进程

- 也称为退出或主程序返回，调用exit可终止进程。exit()向父进程给出一个退出码（8位的整数）
- 父进程终止时如何影响子进程：子进程从父进程继承了进程组ID和终端组ID(控制终端)，因此子进程对发给该进程组或终端组的信号敏感。终端关闭时，以该终端为控制终端的所有进程都收到SIGHUP信号
- 子进程终止时，向父进程发送SIGCHLD信号，父进程截获此信号并通过wait()系统调用来释放子进程PCB

POSIX进程控制

■阻塞和唤醒

■阻塞：

- 暂停一段时间(sleep)
- 暂停并等待信号(pause);
- 等待子进程暂停或终止(wait)

■唤醒：

- 发送信号到某个或一组进程(kill)

POSIX进程控制

- 调用`wait`阻塞本进程以等待子进程的结束，子进程结束时`wait`调用返回
- 父进程创建多个子进程且已有某子进程退出时，父进程中`wait`函数在第一个子进程结束时返回
- 调用格式
 - `pid_t wait(int *status);`
- 返回值为子进程ID

POSIX进程控制

- 调用 `pause` 阻塞本进程以等待信号，接收到信号后恢复执行。当接收到中止进程信号时，该调用不再返回
- 调用格式为
 - `int pause(void);`

POSIX进程控制

- 调用sleep将在指定的时间seconds内阻塞本进程
- 调用格式
 - unsigned sleep(unsigned seconds);
- 返回值为实际的挂起时间

POSIX进程控制

- 调用kill可发送信号sig到某个或一组进程pid
- 调用格式
 - `int kill(pid_t pid, int sig);`
- 信号的定义在文件/usr/ucbinclude/sys/signal.h中
- 命令kill可用于向进程发送信号。如"kill -9 100"将发送SIGKILL到ID为100的进程；该命令将中止该进程的执行

POSIX进程控制

■ **实例：**演示子进程与父进程的关系和fork、exec、wait的使用

■ **两个程序：**

■ main——父进程执行的程序，源代码main.c

■ test——子进程执行的程序，源代码test.c



main.c



test.c

■ **程序main.c**

■ 首先创建第1个子进程

■ 然后进行10次循环，循环到第2次时，创建第2个子进程，循环到第3次时，等待子进程结束

■ **程序test.c**

■ 进行10次循环，在循环中显示当前进程ID和父进程ID

Windows进程控制块

■ Windows的进程由执行体进程对象EPROCESS描述

```
kd> dt _EPROCESS
      +0x000 Pcb                : _KPROCESS
      +0x06c ProcessLock        : _EX_PUSH_LOCK
      +0x070 CreateTime         : _LARGE_INTEGER
      +0x078 ExitTime           : _LARGE_INTEGER
      +0x080 RundownProtect     : _EX_RUNDOWN_REF
      +0x084 UniqueProcessId    : Ptr32 Void
      +0x088 ActiveProcessLinks : _LIST_ENTRY
      +0x0ac PeakVirtualSize    : Uint4B
      .....

```

Windows进程控制块

- 在Windows中，PCB也称为内核进程块KPROCESS，即内核进程对象

```
kd> dt _EPROCESS
      +0x000 Pcb                : _KPROCESS
      +0x06c ProcessLock       : _EX_PUSH_LOCK
      +0x070 CreateTime        : _LARGE_INTEGER
      +0x078 ExitTime          : _LARGE_INTEGER
      +0x080 RundownProtect    : _EX_RUNDOWN_REF
      +0x084 UniqueProcessId   : Ptr32 Void
      +0x088 ActiveProcessLinks : _LIST_ENTRY
      +0x0ac PeakVirtualSize   : Uint4B
      .....

```

Windows进程控制块

■ 与进程相关的内核变量

- PsActiveProcessHead——进程链表头
- PsInitialSystemProcess——系统进程(系统线程的宿主)
- PsIdleProcess——空闲进程

Windows进程控制

- **创建进程：** CreateProcess()函数用于创建新进程及其主线程，以执行指定的程序
 - 新进程可以继承：打开文件的句柄、各种对象(如进程、线程、信号量、管道等)的句柄、环境变量、当前目录、原进程的控制终端、原进程的进程组(用于发送Ctrl+C或Ctrl+Break信号给多个进程)——每个句柄在创建或打开时能指定是否可继承；
 - 新进程不能继承：优先权类、内存句柄、DLL模块句柄

Windows进程控制

■创建进程

```
BOOL CreateProcess(  
    LPCWSTR pszImageName,  
    LPCWSTR pszCmdLine,  
    LPSECURITY_ATTRIBUTES psaProcess,  
    LPSECURITY_ATTRIBUTES psaThread,  
    BOOL fInheritHandles,  
    DWORD fdwCreate,  
    LPVOID pvEnvironment,  
    LPWSTR pszCurDir,  
    LPSTARTUPINFO psiStartInfo,  
    LPPROCESS_INFORMATION pProcInfo  
);
```

Windows进程控制

- **终止进程**：ExitProcess终止一个进程和它的所有线程；它的终止操作是完整的，包括关闭所有对象句柄、它的所有线程等

```
VOID ExitProcess(  
    UINT uExitCode  
);
```

Windows进程控制

- **终止进程**：TerminateProcess()终止指定的进程和它的所有线程；它的终止操作是不完整的（如：不向相关DLL通报关闭情况），通常只用于异常情况下对进程的终止

```
BOOL TerminateProcess(  
    HANDLE hProcess,  
    DWORD uExitCode  
);
```


Windows进程控制

■ 实例

■ 两个程序：

- main——父进程执行的程序，源代码main.c
- test——子进程执行的程序，源代码test.c



main.c



test.c

操作系统，2026

谢谢

OPERATING SYSTEM